

Tema 4. Árboles

Estructuras de Datos
Curso 2024/2025



Índice

4. Árboles

4.1 Definición de árbol y conceptos generales

4.2 TAD árbol

4.3 Árboles binarios

4.4 Implementación de árboles

- Árbol binario mediante listas enlazadas
- Árbol binario basado en array
- Árboles generales

4.5 Recorridos de árboles

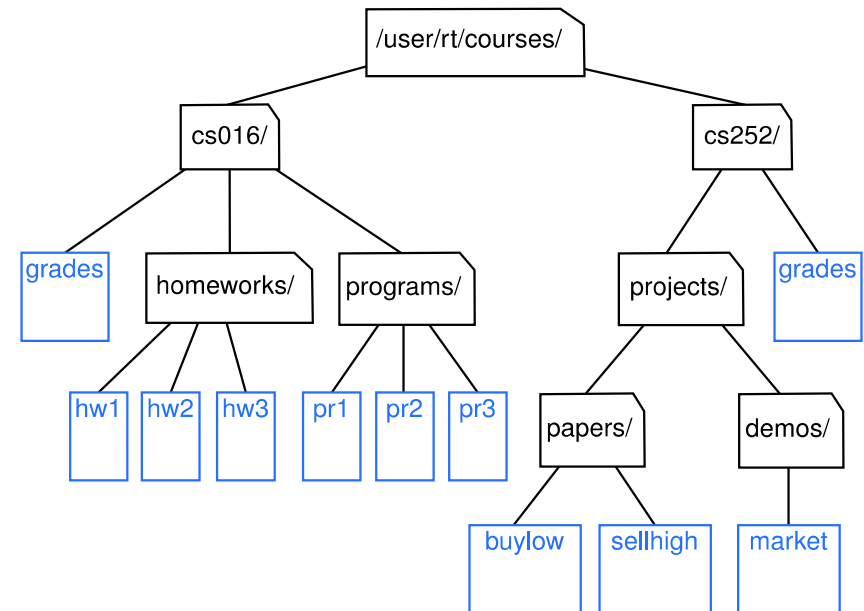
- Preorden
- Postorden
- Primero en anchura
- Enorden
- Aplicaciones

4.6 Búsqueda binaria

4.7 Árboles AVL

Definición de árbol y conceptos generales

- Un árbol es una de las estructuras más importantes en computación
- Proporcionan una **forma natural de organizar información**
- Es una **estructura de datos no lineal** porque tiene relaciones más complejas que un simple "delante de" o "detrás de"
- Las **relaciones en un árbol son jerárquicas**, con objetos por debajo o por encima de otros. Se suele hablar de padres e hijos.



Definición de árbol y conceptos generales

- Salvo el elemento inicial de un **árbol** (conocido como nodo raíz)
 - Cada elemento de un árbol tiene un padre y cero o más hijos
 - Cada elemento de un árbol es un nodo y los nodos están conectados con líneas
- Formalmente, un árbol T es un conjunto de nodos que almacenan elementos, de forma que los nodos tienen una relación padre-hijo que satisface las siguientes propiedades:
 - Si T no está vacío, tiene un nodo especial, llamado *Raíz*, que no tiene padre
 - Cada nodo v diferente del *Raíz* tiene un único nodo padre w
 - Cada nodo con padre w es un hijo de w
- Definición recursiva de un árbol:
 - Un árbol T es un árbol vacío o tiene un nodo r , llamado *Raíz*, y un conjunto de subárboles cuyas raíces son los hijos de r

Definición de árbol y conceptos generales

- Otras **relaciones** importantes en un árbol:
 - Dos nodos que son hijos de un mismo padre son hermanos
 - Un nodo v es externo si no tiene hijos (nodo hoja)
 - Un nodo v es interno si tiene 1 o más hijos
 - Un nodo u es un ancestro de v si es el padre de v o es un ancestro del padre de v
 - Un nodo v es un descendiente de u si u es un ancestro de v
 - Un borde o enlace de un árbol es un par de nodos (u, v) de forma que u es el padre de v o viceversa
 - Un camino en un árbol es una secuencia de nodos de forma que cada dos nodos consecutivos forman un borde o enlace

Definición de árbol y conceptos generales

- La profundidad de un nodo es el número de ancestros que tiene ese nodo. Recursivamente:
 - Si *node* es la raíz, su profundidad es 0, en otro caso la profundidad de *node* es $1 +$ la profundidad del padre de *node*
- La altura de un nodo es definida recursivamente como:
 - Si *node* es una hoja, su altura es 0. En otro caso, la altura de *node* es $1 +$ el máximo de las alturas de los hijos de *node*

Árboles: TAD

- TAD Tree (DESCRIPCIÓN: colección de objetos organizados de forma jerárquica a modo de árbol; VALORES: cualquier objeto; OPERACIONES: CrearTree, root, is_root, parent, num_children, children, is_leaf, len, is_empty, nodes, iter, depth, height)

Árboles: TAD

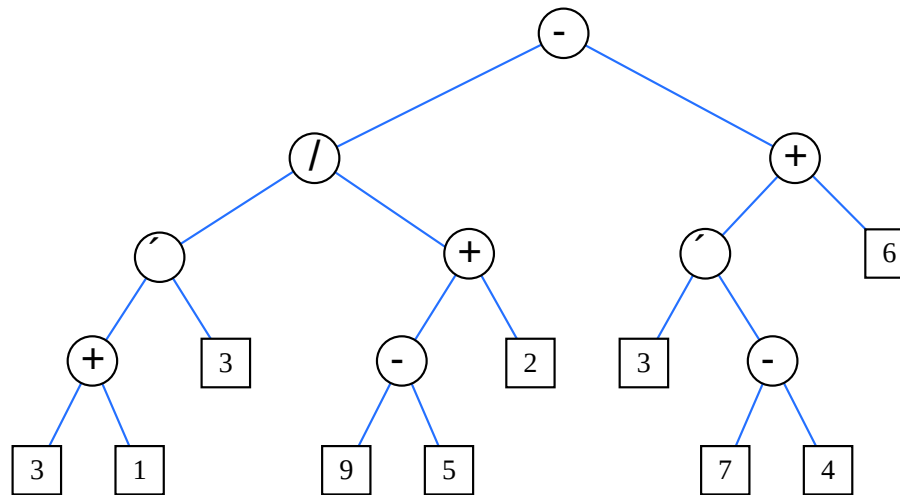
- Si T es un árbol:
 - `CrearTree` – Crear un árbol vacío
 - `T.root()` – devuelve el nodo raíz del árbol. *None* si está vacío
 - `T.is_root(node)` – devuelve *True* si *node* es la raíz del árbol
 - `T.parent(node)` – devuelve el nodo padre del nodo *node*
 - `T.num_children(node)` – devuelve el número de hijos del nodo *node*
 - `T.children(node)` – iteración sobre los hijos del nodo *node*
 - `T.is_leaf(node)` – devuelve *True* si *node* no tiene hijos
 - `len(T)` – devuelve el número de nodos en T
 - `T.is_empty()` – devuelve *True* si T está vacío.
 - `T.nodes()` – itera por todos los nodos de T
 - `iter(T)` – itera por los elementos almacenados en T
 - `T.depth(node)` – devuelve la profundidad del nodo *node*
 - `T.height(node)` – devuelve la altura del nodo *node*

Árboles binarios: conceptos

- Es un árbol con las siguientes **propiedades**:
 - Cada nodo tiene como mucho 2 hijos
 - Cada nodo hijo es etiquetado como hijo izquierdo e hijo derecho
 - Un hijo izquierdo precede a un hijo derecho dentro los hijos de un nodo
 - Un árbol binario es propio si cada nodo tiene 0 o dos hijos (árbol binario completo)
 - Un árbol binario que no es propio es impropio

Árboles binarios: conceptos

- Ejemplo de árbol binario:
 - Una expresión aritmética puede ser representada como un árbol binario cuyas hojas son variables o constantes y los nodos internos están asociados con operadores $+$, $-$, $\times$, $/$



- El árbol representa la expresión
$$(((3+1) \times 3) / ((9-5)+2)) - ((3 \times (7-4)) + 6))$$
- El árbol es propio porque los operadores $+$, $-$, $\times$, $/$ siempre tienen dos operandos

Árboles binarios: conceptos

- Un árbol binario puede ser definido de forma recursiva:
 - Un árbol binario es un árbol vacío o está formado por:
 - Un nodo r , llamado raíz, que almacena un elemento
 - Un árbol binario (posiblemente vacío), que es el subárbol izquierdo de T
 - Un árbol binario (posiblemente vacío), que es el subárbol derecho de T

Árboles binarios: TAD

- Además de las operaciones del TAD Tree, el árbol binario añade otras operaciones
- TAD BinaryTree (DESCRIPCIÓN: colección de objetos organizados de forma jerárquica a modo de árbol donde cada nodo tiene como mucho dos hijos; VALORES: cualquier objeto; OPERACIONES: CrearArbol, root, is_root, parent, num_children, children, is_leaf, len, is_empty, nodes, iter, depth, height, **left, right, sibling, add_root, add_left, add_right, replace, delete, attach**)

Árboles binarios: TAD

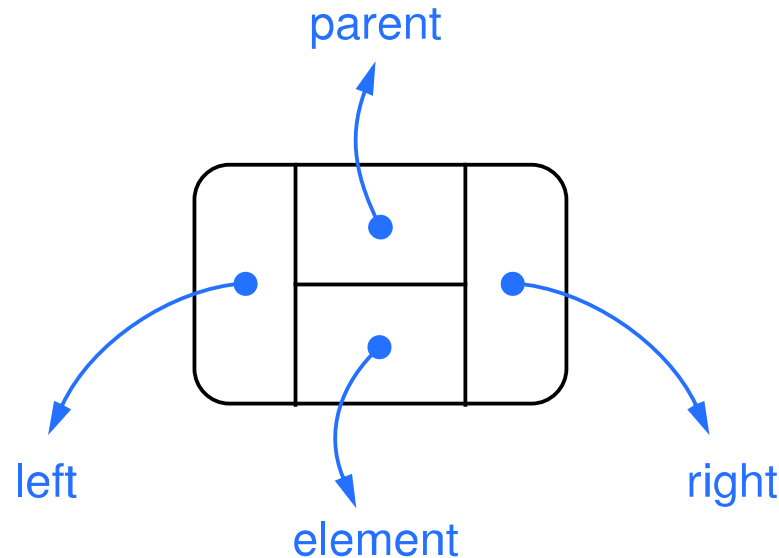
- Además de las operaciones del TAD Tree, el árbol binario añade las siguientes:
 - T.left(node) – devuelve el hijo izquierdo de *node* o *None* si no existe
 - T.right(node) – devuelve el hijo derecho de *node* o *None* si no existe
 - T.sibling(node) – devuelve el nodo hermano de *node* o *None* si no existe
 - T.add_root(e) – crea el nodo raíz de un árbol vacío y almacena en él el elemento *e*. Devuelve el nodo resultante. Se produce un error si ya existe un nodo raíz
 - T.add_left(node,e) – crea un nodo nuevo para almacenar *e* y lo sitúa como hijo izquierdo de *node*. Devuelve el nodo resultante. Se produce un error si *node* ya tiene un hijo izquierdo
 - T.add_right(node,e) – crea un nodo nuevo para almacenar *e* y lo sitúa como hijo derecho de *node*. Devuelve el nodo resultante. Se produce un error si *node* ya tiene un hijo derecho

Árboles binarios: TAD

- $T.replace(node, e)$ – reemplaza el elemento de $node$ por e y devuelve el elemento previamente almacenado en $node$
- $T.delete(node)$ – elimina el nodo $node$ y lo sustituye con su hijo si tiene alguno. Devuelve el elemento almacenado en $node$. Error si tiene dos hijos
- $T.attach(node, T1, T2)$ – incorpora $T1$ y $T2$ como hijos izquierda y derecha de $node$, y establece $T1$ y $T2$ como árboles vacíos. Error si $node$ no es una hoja

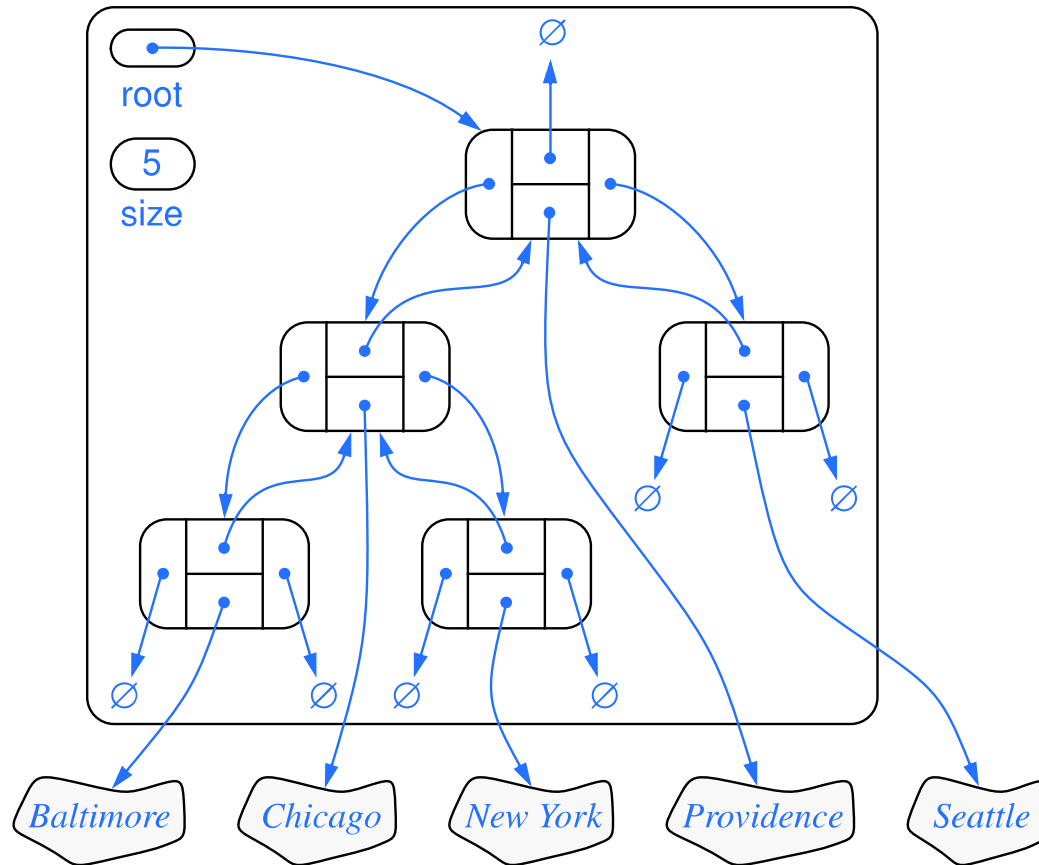
Árboles binarios: implementación

- Veremos una **implementación basada en listas enlazadas**:
 - Un nodo del árbol tiene la siguiente estructura



Árboles binarios: implementación

- Un árbol tiene la siguiente estructura:



Árboles binarios: implementación

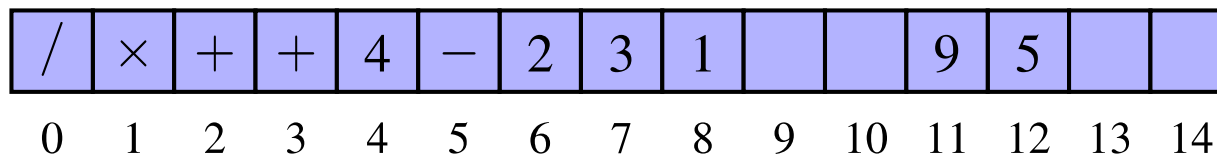
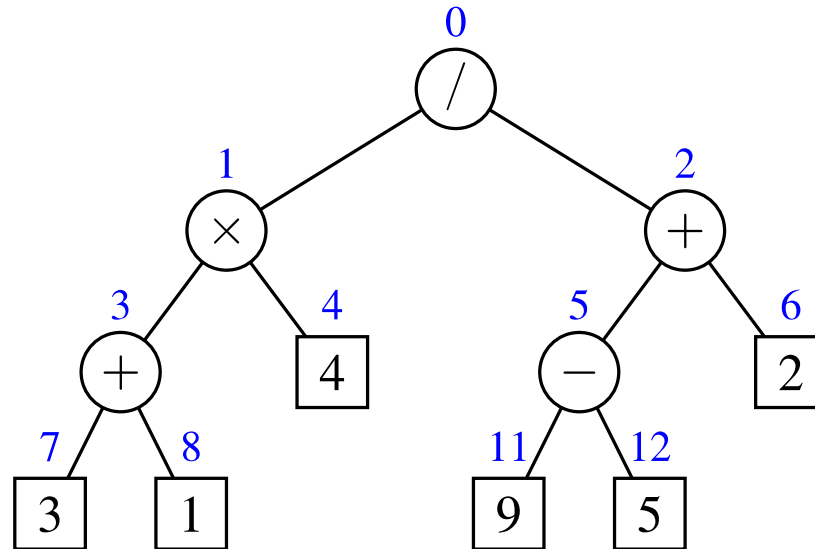
- Implementación Python en fichero `linked_binary_tree.py`

Árboles binarios: implementación

- Podemos desarrollar una implementación de un TAD árbol donde el **árbol es representado mediante un array**:
 - Los elementos se guardan en una estructura de datos secuencial
 - Para cada nodo *node* del árbol T , $f(node)$ es un entero definido como:
 - Si *node* es el nodo raíz, $f(node) = 0$
 - Si *node* es el hijo izquierdo de *u*, entonces $f(node) = 2f(u) + 1$
 - Si *node* es el hijo derecho de *u*, entonces $f(node) = 2f(u) + 2$
 - El elemento en el nodo *node* será almacenado en el índice $f(node)$ del array

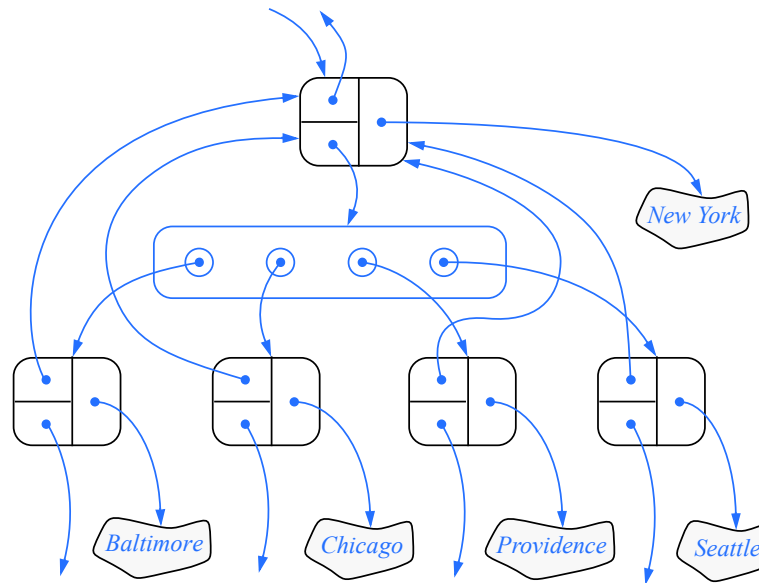
Árboles binarios: implementación

- Ejemplo de un árbol implementado como array que almacena una expresión:



Árboles generales: implementación

- En un **árbol general**, no hay un límite para el número de hijos de un nodo
- Para implementar un **árbol general usando listas enlazadas**, cada nodo debe tener un contenedor de referencias a sus hijos (por ejemplo, una lista de referencias a los hijos):



Árboles binarios: eficiencia

- Eficiencia de la **implementación que hemos visto de árbol binario con listas enlazadas**
 - Todas las operaciones son $O(1)$, salvo aquellas que implican recorridos

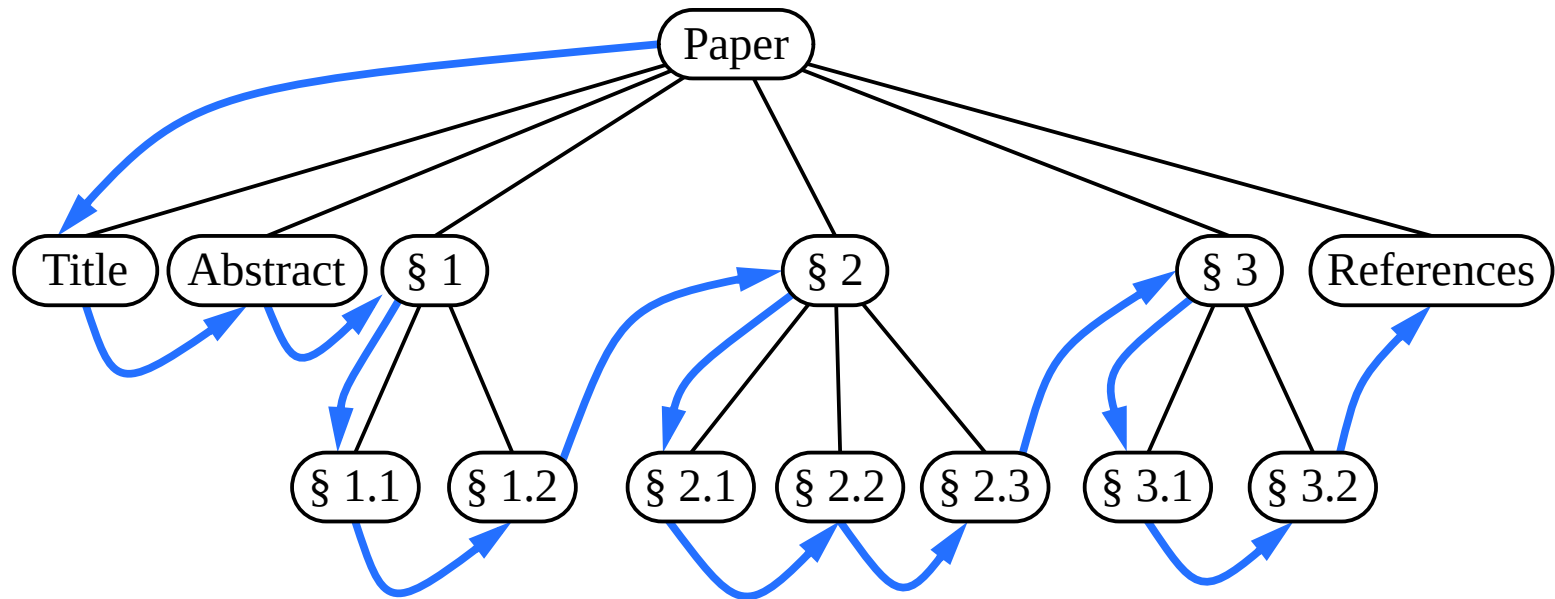
Operación	Eficiencia
len, is_empty	$O(1)$
root, parent, left, right, sibling, children, num_children,	$O(1)$
is_root, is_leaf	$O(1)$
depth(node)	$O(n)$
height(node)	$O(n)$
add_root, add_left, add_right, replace, delete, attach	$O(1)$
recorridos	$O(n)$

Árboles generales: recorridos

- Un **recorrido de un árbol** es una forma sistemática de visitar todos los nodos del árbol.
- Veremos distintas formas de hacerlo
 - Preorden
 - Postorden
 - Primero en anchura
 - Enorden (específico de árboles binarios)

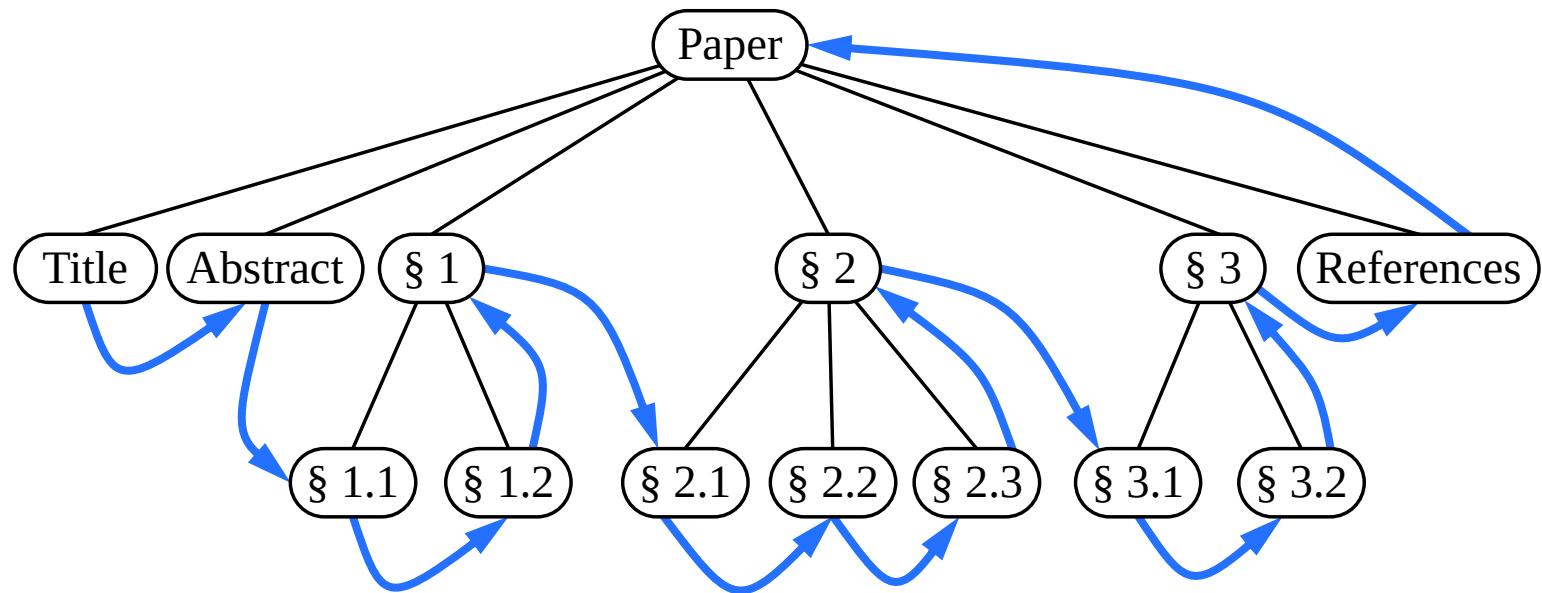
Árboles generales: recorridos

- **Preorden:** la raíz es visitado primero y después son visitados recursivamente de izquierda a derecha los subárboles que son sus hijos



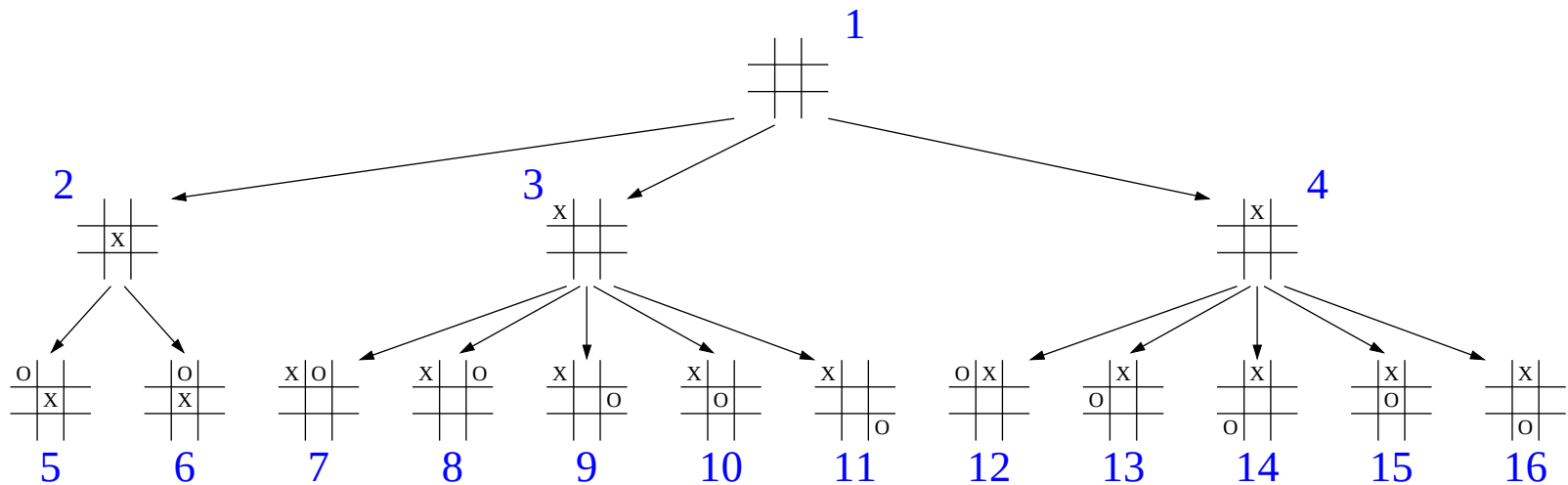
Árboles generales: recorridos

- **Postorden:** primero visita recursivamente los subárboles de los hijos del nodo y después el propio nodo



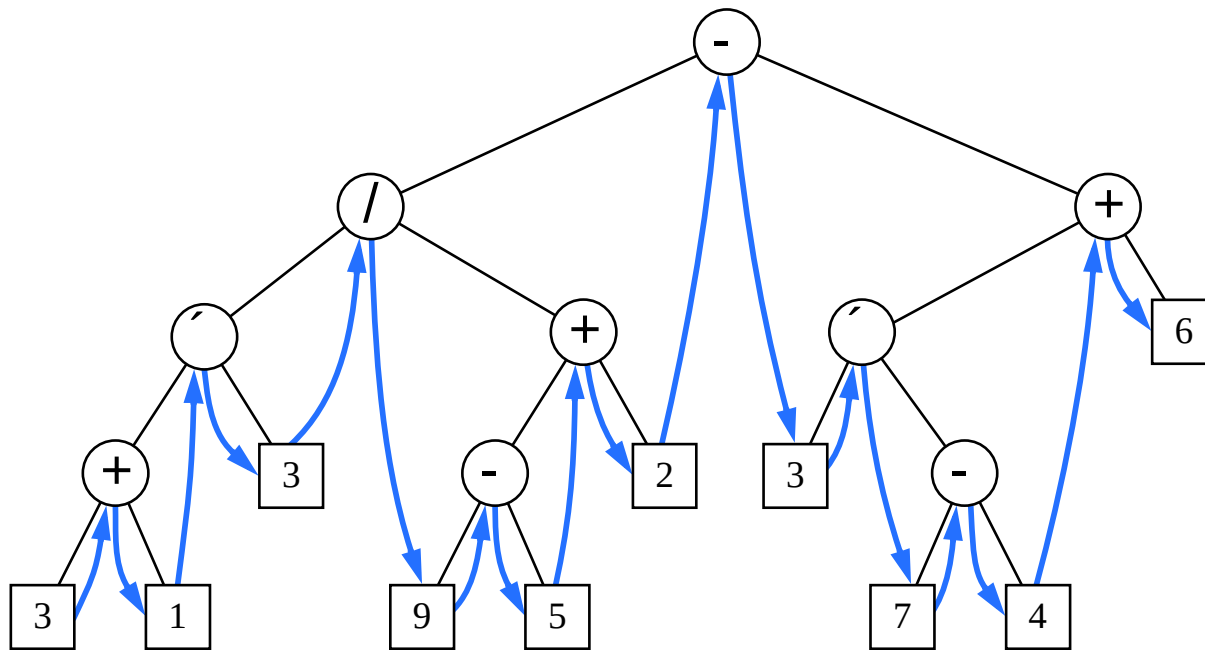
Árboles generales: recorridos

- **Primero en anchura:** visitar las posiciones de un nivel de profundidad antes de pasar al nivel siguiente. El proceso no es recursivo.



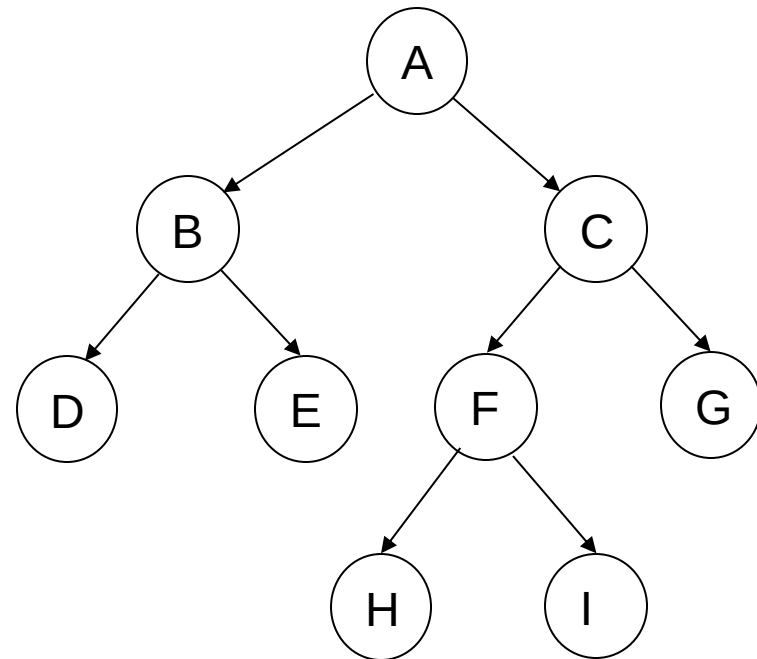
Árboles binarios: recorridos

- **Enorden:** recorrido específico para un árbol binario. Se visita el propio nodo entre las visitas recursivas de los subárboles izquierda y derecha. Se puede ver como la visita de los nodos de T de izquierda a derecha.



Árboles binarios: ejercicio de recorridos

- Dado el árbol siguiente, indicar el orden en el que son visitados los nodos mediante preorden, enorden, postorden y primero en anchura



Árboles binarios: ejercicio de recorridos

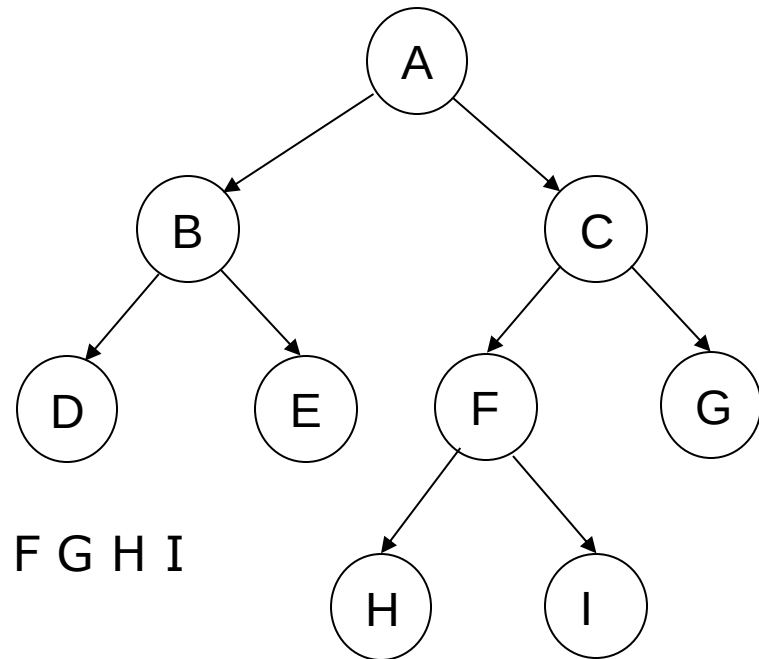
- Dado el árbol siguiente, indicar el orden en el que son visitados los nodos mediante preorden, enorden, postorden y primero en anchura

Preorden: A B D E C F H I G

Enorden: D B E A H F I C G

Postorden: D E B H I F G C A

Primero en anchura: A B C D E F G H I



Árboles binarios: implementación de recorridos

Preorden

```
def __iter__(self):
    for node in self.nodes():
        yield node.element()

def nodes(self):
    return self.preorder() # se podría seleccionar otro recorrido

def preorder(self):
    if not self.is_empty():
        for node in self._subtree_preorder(self.root()):
            yield node

def _subtree_preorder(self, node):
    yield node
    for c in self.children(node):
        for other in self._subtree_preorder(c):
            yield other
```

Árboles binarios: implementación de recorridos

Postorden

```
def postorder(self):  
    if not self.is_empty():  
        for node in self._subtree_postorder(self.root()):  
            yield node  
  
def _subtree_postorder(self, node):  
    for c in self.children(node):  
        for other in self._subtree_postorder(c):  
            yield other  
    yield node
```

Árboles binarios: implementación de recorridos

Primero en anchura

```
def breadthfirst(self):  
    if not self.is_empty():  
        q = ArrayQueue()  
        q.enqueue(self.root())  
        while not q.is_empty():  
            node = q.dequeue()  
            yield node  
            for c in self.children(node):  
                q.enqueue(c)
```

Árboles binarios: implementación de recorridos

Enorden

```
def inorder(self):  
    if not self.is_empty():  
        for node in self._subtree_inorder(self.root()):  
            yield node  
  
def _subtree_inorder(self, node):  
    if self.left(node) is not None:  
        for other in self._subtree_inorder(self.left(node)):  
            yield other  
    yield node  
    if self.right(node) is not None:  
        for other in self._subtree_inorder(self.right(node)):  
            yield other
```

Árboles generales: aplicaciones de recorridos

- Impresión de los elementos de un árbol: Tabla de contenidos

Paper

Title

Abstract

Section 1

Subsection 1.1

Subsection 1.2

Section 2

...

```
def escritura_arbol(T,node,d):  
    print(2*d*' ' +str(node.element()))  
    for c in T.children(node):  
        escritura_arbol(T,c,d+1)
```

Árboles binarios: aplicaciones de recorridos

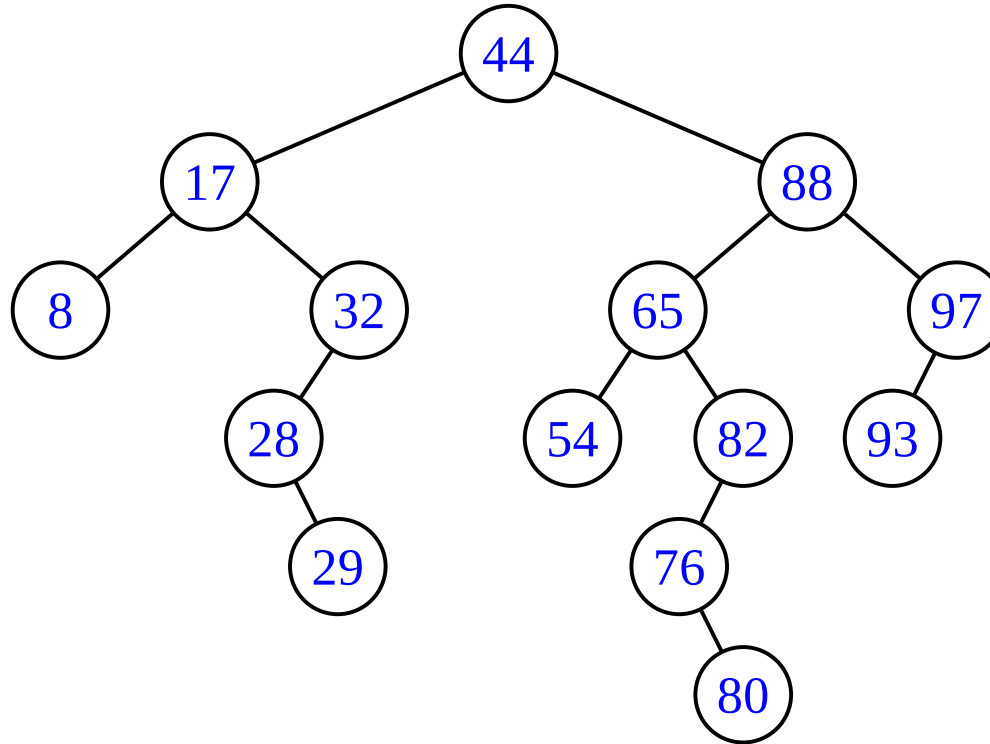
- Desarrollar una función que permita imprimir un árbol binario desde un fichero. Si uno de los hijos no existe debe aparecer como None en el lugar adecuado. Se debe usar la indentación apropiada para mostrar las distintas profundidades de los elementos del árbol.
- Fichero EscrituraArbol.py

Árboles binarios: búsqueda binaria

- Una aplicación importante de un árbol binario es como un **árbol de búsqueda**
- Lo usaremos para implementar de forma eficiente un mapa ordenado asumiendo que las claves tienen una relación de orden
- Un **árbol de búsqueda binaria** es un árbol binario T donde cada nodo *node* guarda un par clave-valor (k,v) cumpliendo que:
 - Las claves del subárbol izquierda de *node* son menores que k
 - Las claves del subárbol derecha de *node* son mayores que k

Árbol de búsqueda binaria

- Un árbol de búsqueda binaria cumple la propiedad de que el recorrido en orden del árbol visita las posiciones en orden creciente de sus claves



Árbol de búsqueda binaria: TAD

- TAD TreeMap (DESCRIPCIÓN: estructura de datos que asocia claves únicas con valores, donde las claves tienen una relación de orden; VALORES: colección de (clave,valor); OPERACIONES: CrearTreeMap, root, is_root, parent, num_children, children, is_leaf, len, is_empty, nodes, iter, depth, height, left, right, sibling, add_root, add_left, add_right, replace, delete, attach, **first, last, before, after, find_node, find_min, find_max, find_le, find_lt, find_ge, find_gt, find_range**, getitem, setitem, delitem)
- Además de las operaciones del TAD BinaryTree, el árbol de búsqueda binaria añade las siguientes, siendo TM un árbol de búsqueda binaria:
 - TM.first() – Devuelve el nodo con la menor clave o None si está vacío
 - TM.last() – Devuelve el nodo con la mayor clave o None si está vacío.
 - TM.before(node) – Devuelve el nodo que está delante del nodo *node* en la relación de orden. None si *node* es la primera posición
 - TM.after(node) – Devuelve el nodo que está detrás del nodo *node* en la relación de orden. None si *node* es la última posición

Árbol de búsqueda binaria

- `find_node(k)` – Devuelve el nodo con la clave k o el último nodo examinado si no está. *None* si está vacío.
- `find_min()` – Devuelve clave-valor con la menor clave. *None* si está vacío.
- `find_max()` – Devuelve clave-valor con la mayor clave. *None* si está vacío.
- `find_le(k)` – Devuelve clave-valor con la mayor clave menor o igual que k . *None* si está vacío.
- `find_lt(k)` – Devuelve clave-valor con la mayor clave menor que k . *None* si está vacío.
- `find_ge(k)` – Devuelve clave-valor con la menor clave mayor o igual que k . *None* si está vacío.
- `find_gt(k)` – Devuelve clave-valor con la menor clave mayor que k . *None* si está vacío.
- `find_range(start, stop)` – Itera clave-valor tal que $\text{start} \leq \text{clave} < \text{stop}$
- Como el árbol de búsqueda binaria implementa también operaciones de los mapas (diccionarios), se añaden las siguientes:
 - `M[k]` – Devuelve el valor v asociado a la clave k (método `__getitem__` en Python). Error si no existe
 - `M[k] = v` – Asocia el valor v a la clave k . Si ya existe reemplaza el valor (método `__setitem__` en Python)
 - `del M[k]` – Elimina el *item* con clave k . Error si no existe. (método `__delitem__` en Python)

Árbol de búsqueda binaria: TAD

- En el árbol binario habíamos definido métodos para movernos en el árbol como $parent(n)$, $left(n)$ y $right(n)$.
Añadimos **métodos para movimientos** según orden de las claves:
 - $T.first()$ - devuelve el nodo con la menor clave o *None* si T está vacío.
 - $T.last()$ - devuelve el nodo con la mayor clave o *None* si T está vacío.
 - $T.before(node)$ - devuelve el nodo con la clave menor que *node* o *None* si *node* tiene la clave más pequeña.
 - $T.after(node)$ - devuelve el nodo con la clave mayor que *node* o *None* si *node* tiene la clave más grande.

Árbol de búsqueda binaria: posición posterior

Algoritmo after(node):

if right(node) is not None **then**

 walk = right(node)

while left(walk) is not None **do**

 walk = left(walk)

return walk

else

 walk = node

 ancestor = parent(walk)

while ancestor is not None **and** walk == right(ancestor) **do**

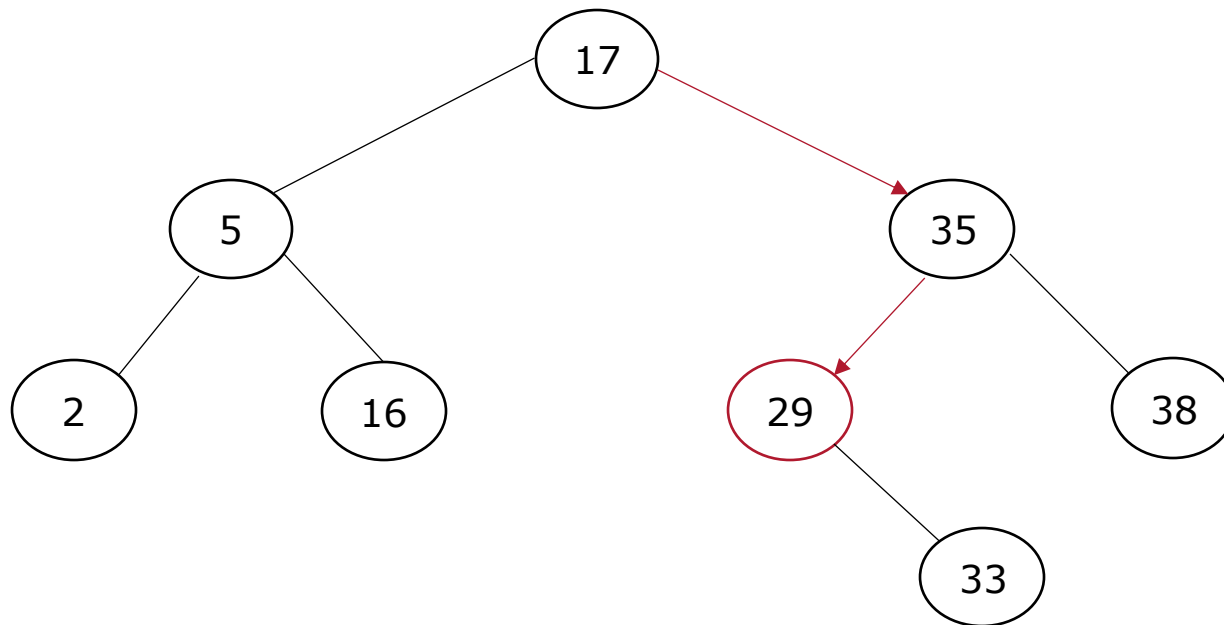
 walk = ancestor

 ancestor = parent(walk)

return ancestor

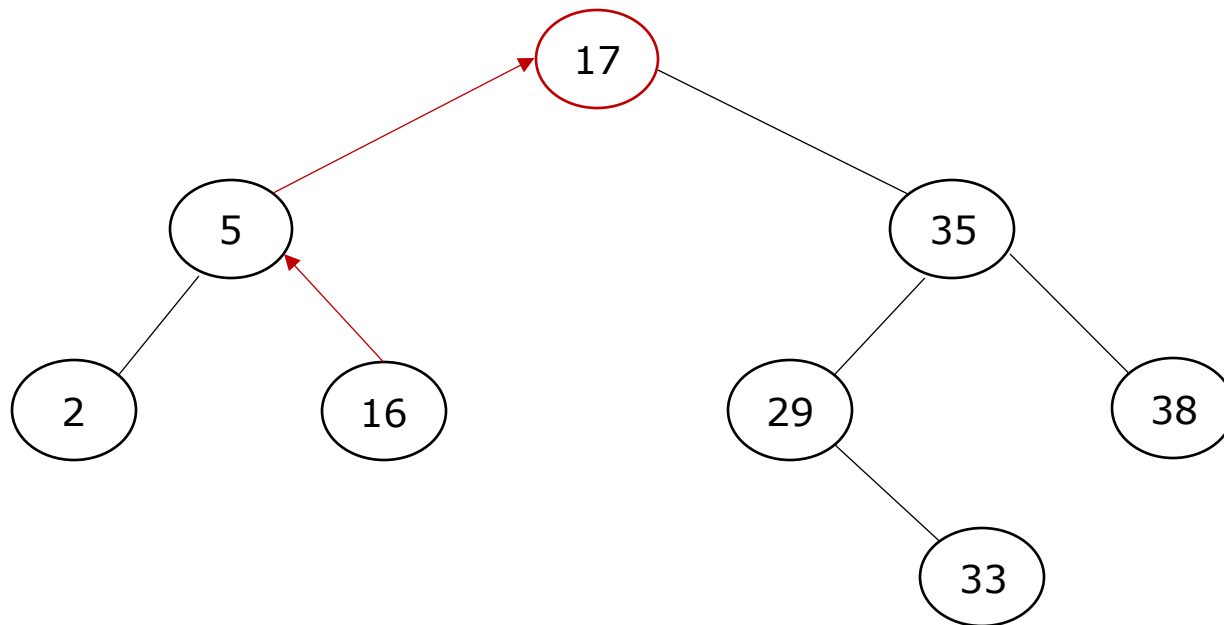
Árbol de búsqueda binaria: posición posterior

- Por ejemplo, $after(node)$ donde $node$ contiene la clave 17 (tiene hijo derecho) supone tomar la rama derecha de $node$ y luego la rama izquierda sucesivamente hasta alcanzar *None*, devolviendo el nodo de 29



Árbol de búsqueda binaria: posición posterior

- Por ejemplo, *after(node)* donde *node* contiene la clave 16 (NO tiene hijo derecho) supone subir en el árbol hasta que el nodo de recorrido deje de ser un hijo derecho de su padre, y entonces devolver el padre



Árbol de búsqueda binaria: posición anterior

Algoritmo before(node):

if left(node) is not None **then**

walk = left(node)

while right(walk) is not None **do**

walk = right(walk)

return walk

else

walk = node

ancestor = parent(walk)

while ancestor is not None **and** walk == left(ancestor) **do**

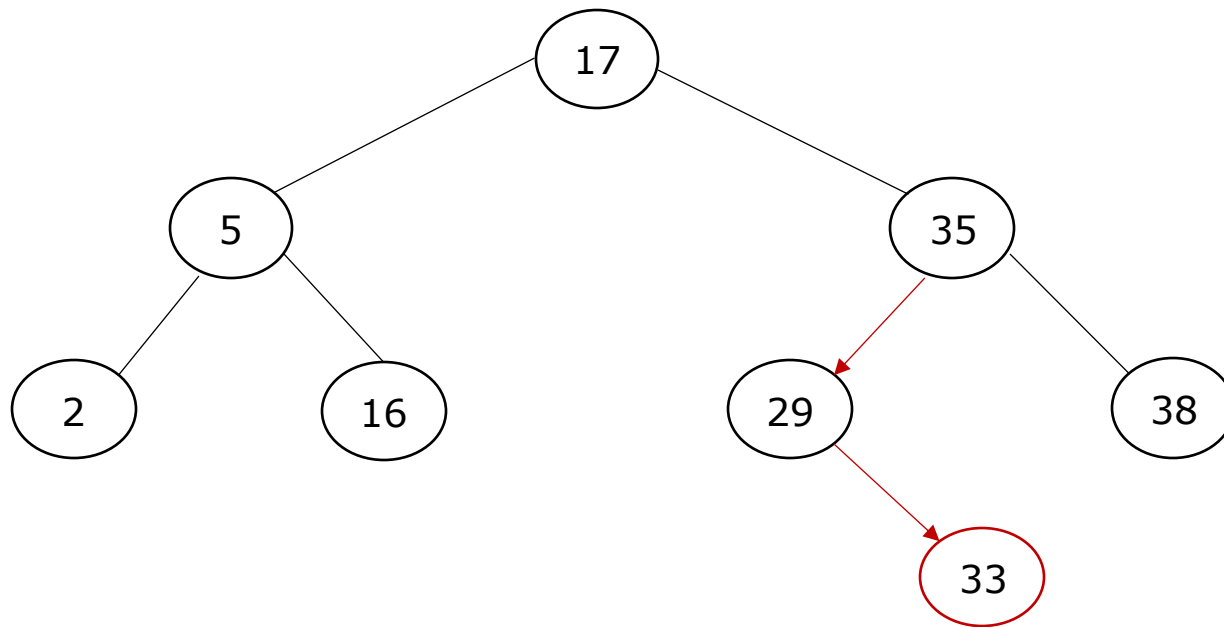
walk = ancestor

ancestor = parent(walk)

return ancestor

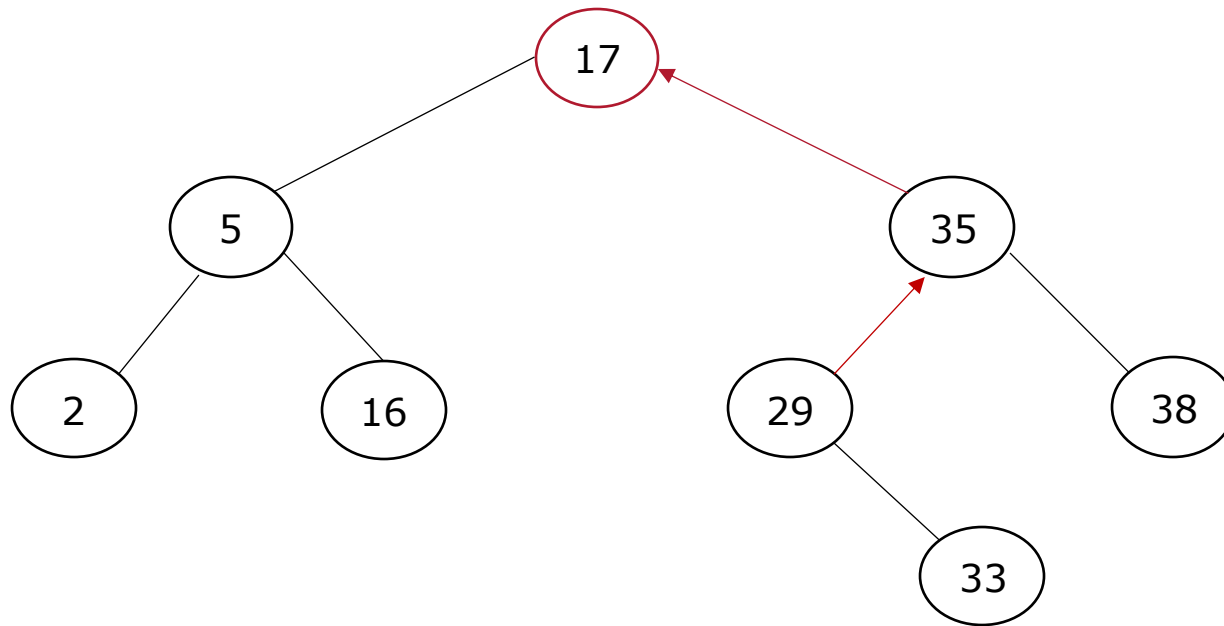
Árbol de búsqueda binaria: posición anterior

- Por ejemplo, *before(node)* donde *node* contiene la clave 35 (tiene hijo izquierdo) supone tomar la rama izquierda de *node* y luego la rama derecha sucesivamente hasta alcanzar *None*, devolviendo el nodo de 33



Árbol de búsqueda binaria: posición anterior

- Por ejemplo, *before(node)* donde *node* contiene la clave 29 (NO tiene hijo izquierdo) supone tomar el padre de *node* y luego sucesivamente nos movemos tomando el padre si el nodo (35) es hijo izquierdo o hasta alcanzar *None*, devolviendo el nodo de 17



Árbol de búsqueda binaria: búsqueda

- La consecuencia más importante de la estructura de un árbol de búsqueda binaria es el **algoritmo de búsqueda**, que se lleva a cabo comparando la clave buscada con la clave almacenada en el nodo:

Algoritmo TreeSearch(T, node, k):

if k == node.key() **then**

return node {elemento encontrado}

else if k < node.key() and T.left(node) **is not** None **then**

return TreeSearch(T, T.left(node), k)

else if k > node.key() and T.right(node) **is not** None **then**

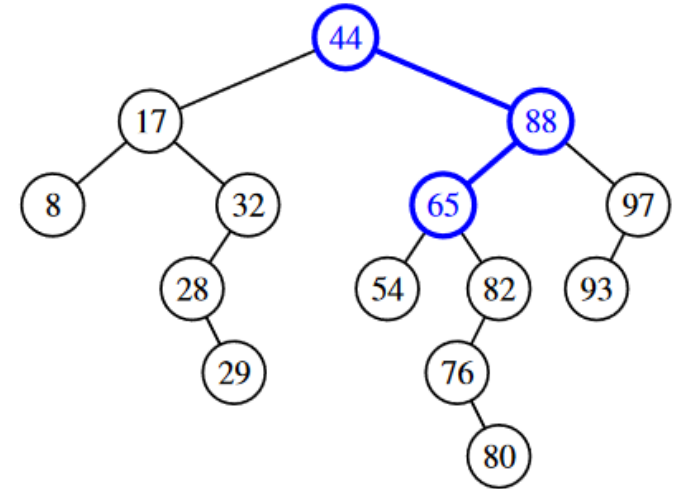
return TreeSearch(T, T.right(node), k)

return node

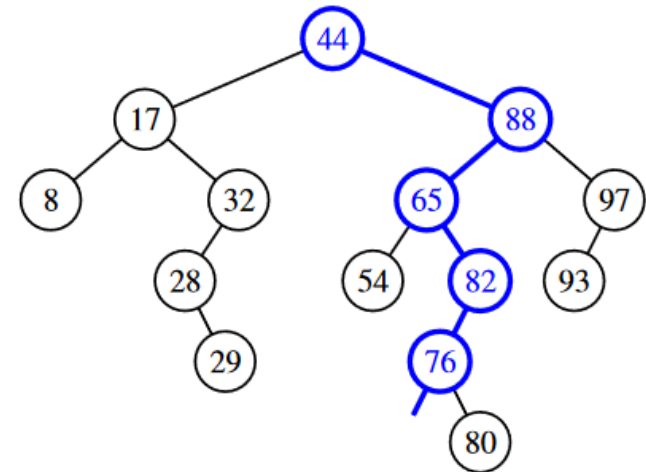
- Si se encuentra la clave, devuelve el nodo que la contiene, y si no la encuentra, devuelve el último nodo explorado. Se utilizará para la implementación de `__getitem__`

Árbol de búsqueda binaria: búsqueda

- Ejemplo de búsqueda con éxito de la clave 65



- Ejemplo de búsqueda sin éxito de la clave 68



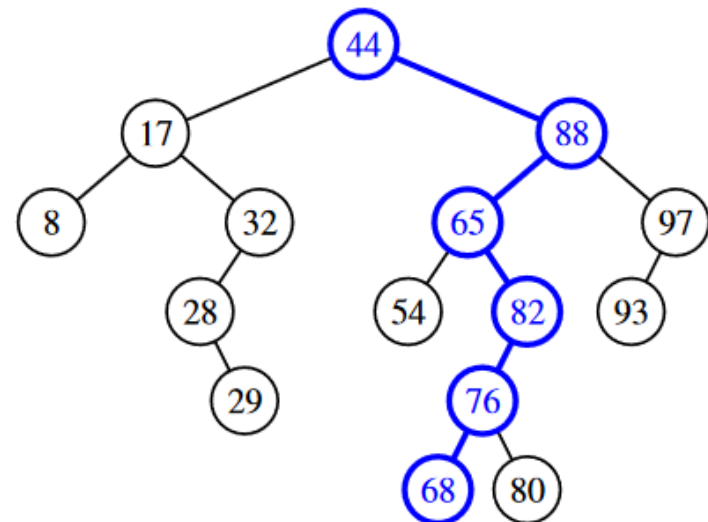
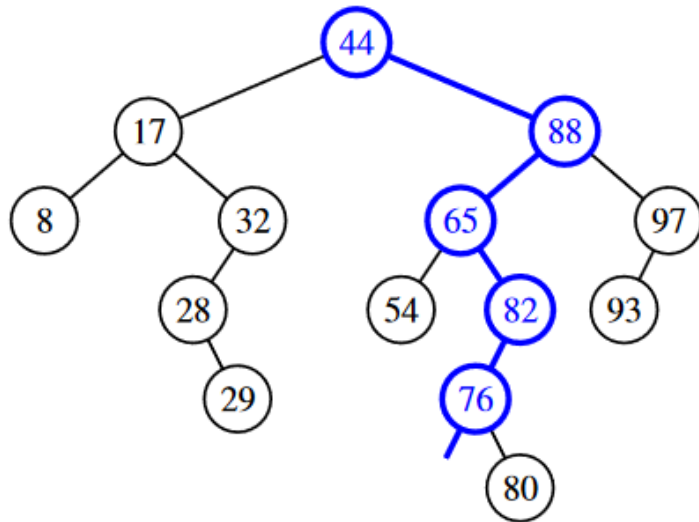
Árbol de búsqueda binaria: asignación de elemento

- Para insertar un elemento en el árbol M mediante $M[k]=v$ (`__setitem__`):
 - Realiza una búsqueda (binaria) de k :
 - Si se encuentra, se le asigna el nuevo valor v
 - Si no se encuentra, se inserta un nuevo elemento en el último nodo alcanzado durante la búsqueda

Algoritmo TreeInsert(T, k, v):
 $node = \text{TreeSearch}(T, T.\text{root}(), k)$
 if $k == node.\text{key}()$ **then**
 Establece v como valor de $node$
 else if $k < node.\text{key}()$ **then**
 Inserta item (k, v) como hijo izquierdo de $node$
 else
 Inserta item (k, v) como hijo derecho de $node$

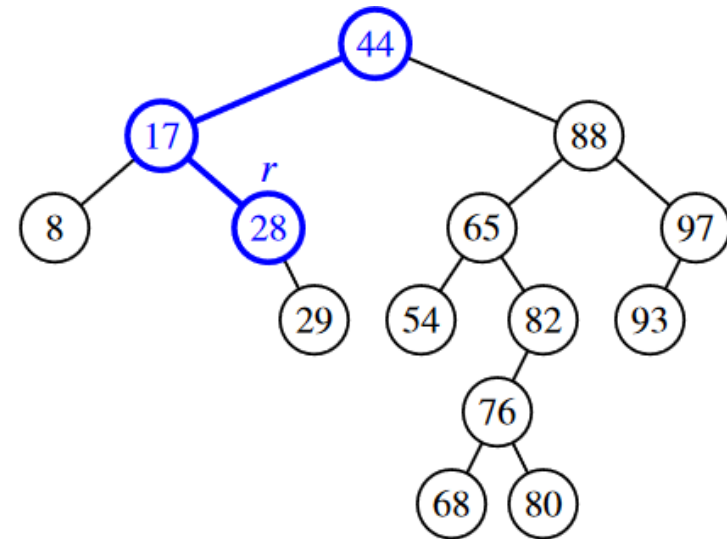
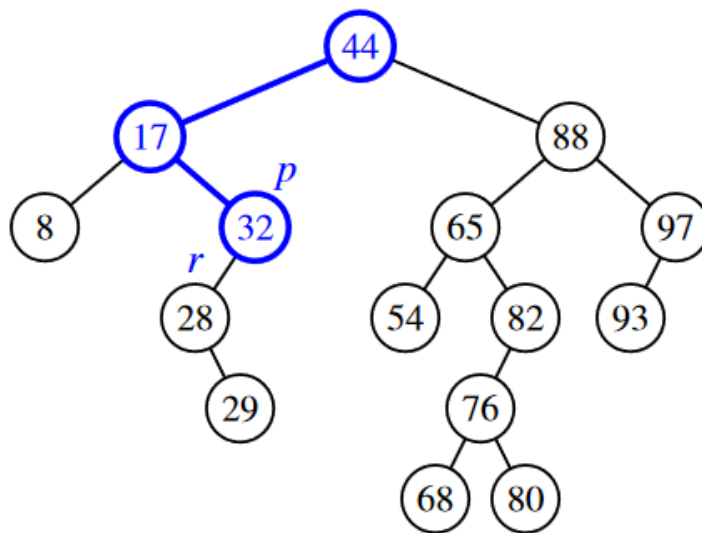
Árbol de búsqueda binaria: asignación de elemento

- Por ejemplo, insertamos la clave 68:



Árbol de búsqueda binaria: eliminación de elemento

- Para eliminar un elemento del árbol con clave k , primero lo buscamos para encontrar su nodo $node$ y, si está, hay dos casos:
 - Caso 1: Si $node$ tiene como mucho un hijo, se borra $node$ y se sustituye por su hijo, preservando las propiedades del árbol de búsqueda. Por ejemplo, borramos el nodo con $k = 32$

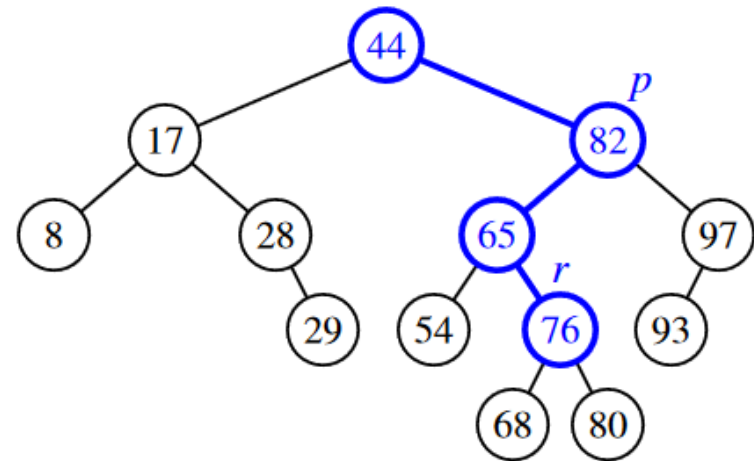
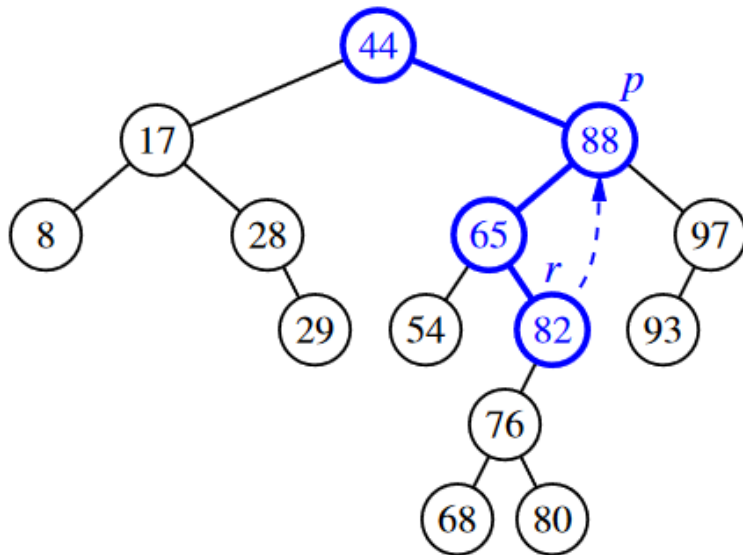


Árbol de búsqueda binaria: eliminación de elemento

- Caso 2: si *node* tiene dos hijos:
 1. Buscamos el nodo *r* que tiene la clave anterior a la de *node* ($r = \text{before}(\text{node})$)
 2. El nodo *r* será usado como sustituto del borrado *node*. Todas las claves en el subárbol derecho de *node* serán mayores que *r* y todas las claves en el subárbol izquierdo de *node* serán menores que *r*, por lo que el árbol sigue verificando la propiedad de árbol de búsqueda binaria
 3. Como *r* sustituye a *node*, *r* debe ser borrado de su antigua posición. Dado que *r* es el nodo más a la derecha del subárbol izquierda de *node*, no tiene hijo derecho y podemos borrarlo de acuerdo con el caso 1.
- Usaremos este algoritmo para implementar `__delitem__`

Árbol de búsqueda binaria: eliminación de elemento

- Por ejemplo, si borramos el nodo con clave $k = 88$



Árbol de búsqueda binaria: Implementación

- Implementación Python en fichero `binary_search_tree.py`

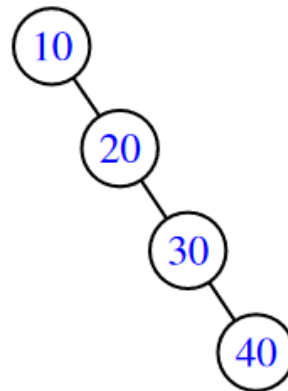
Árbol de búsqueda binaria: eficiencia

- Un árbol de búsqueda binaria T es una implementación eficiente de un mapa de n valores solo si la altura es pequeña, ya que la ejecución de la mayoría de las operaciones es $O(h)$ donde h es la altura del árbol

Operación	Eficiencia
$k \text{ in } T$	$O(h)$
$T[k] = v$	$O(h)$
del $T[k]$	$O(h)$
$T.\text{before}(\text{node}), T.\text{after}(\text{node})$	$O(h)$
$\text{iter}(T)$	$O(n)$

Árbol de búsqueda binaria: eficiencia

- Si podemos mantener la altura h con un valor pequeño, $h \approx \log_2(n)$
 - Ejecución de las operaciones es $O(\log n)$
 - Las operaciones se ejecutan en un tiempo logarítmico
 - En el peor caso, $h=n$:
 - Las operaciones se ejecutan en $O(n)$
 - Se pierde la eficiencia de esta implementación
 - Esto ocurre cuando se insertan claves en orden creciente o decreciente:

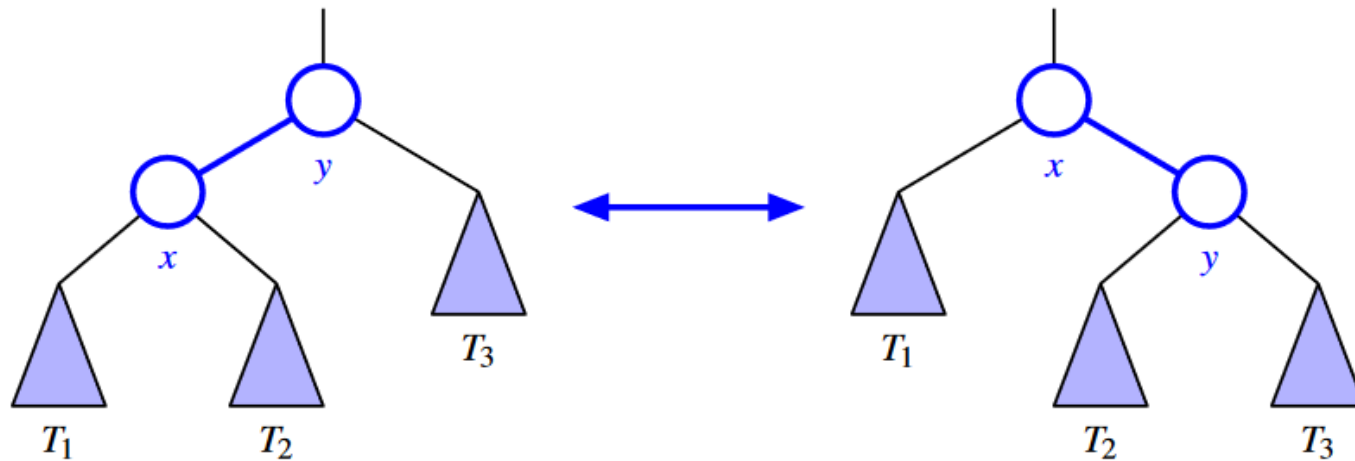


Árbol de búsqueda binaria: árboles balanceados

- En un **árbol de búsqueda balanceado** la diferencia de altura de los hijos de cada nodo es 0 o 1
 - Si no se cumple la condición, se dice que está desbalanceado
- Si tenemos un árbol de búsqueda binario balanceado, las operaciones serán $O(\log n)$
- Vamos a aumentar el TAD árbol de búsqueda binaria con operaciones que se ejecutan ocasionalmente para reestructurar el árbol y disminuir su altura:
 - Rotación
 - Reestructuración trinodo

Árbol de búsqueda binaria: árboles balanceados

- La operación básica se llama **rotación**, que hace que un hijo esté por delante de su padre (posiciones que rotan)



- Se mantiene que las claves en T_1 son menores que x , las claves en T_2 están entre x e y ; y las claves de T_3 son mayores que y
- Se deben reconectar de nuevo las claves que estaban entre las dos posiciones que rotan (T_2)
- Al pasar de la primera figura a la segunda, T_1 disminuye su profundidad y T_3 aumenta su profundidad
- La rotación tiene un tiempo de ejecución de $O(1)$

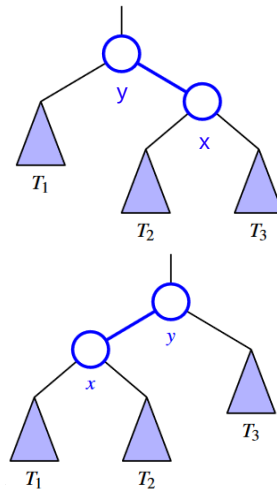
Árbol de búsqueda binaria: árboles balanceados

- Se pueden combinar una o más rotaciones. Una combinación que usaremos es la **reestructuración trinodo**
 - Consideraremos tres nodos: x , y , z , donde $y = \text{padre}(x)$ y $z = \text{padre}(y)$. El objetivo es reestructurar el subárbol que parte de z para disminuir la profundidad de x

Algoritmo rotate(T , node):

```

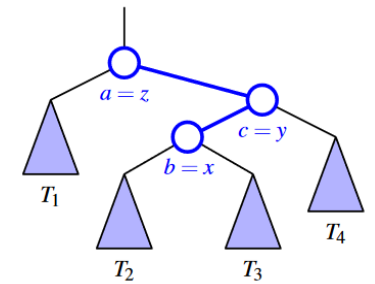
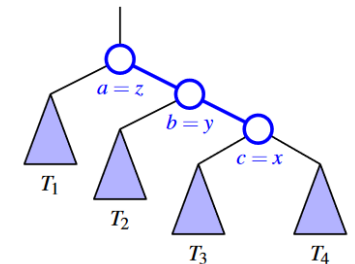
x = node
y = T.parent(x)
z = T.parent(y)
if z is None then
    T.make_root(x)
else if y == T.left(z)
    T.make_left(z, x)
else
    T.make_right(z, x)
if x == T.left(y)
    T.make_left(y, T.right(x)),
    T.make_right(x, y)
else
    T.make_right(y, T.left(x))
    T.make_left(x, y)
  
```



Algoritmo restructure(T , x):

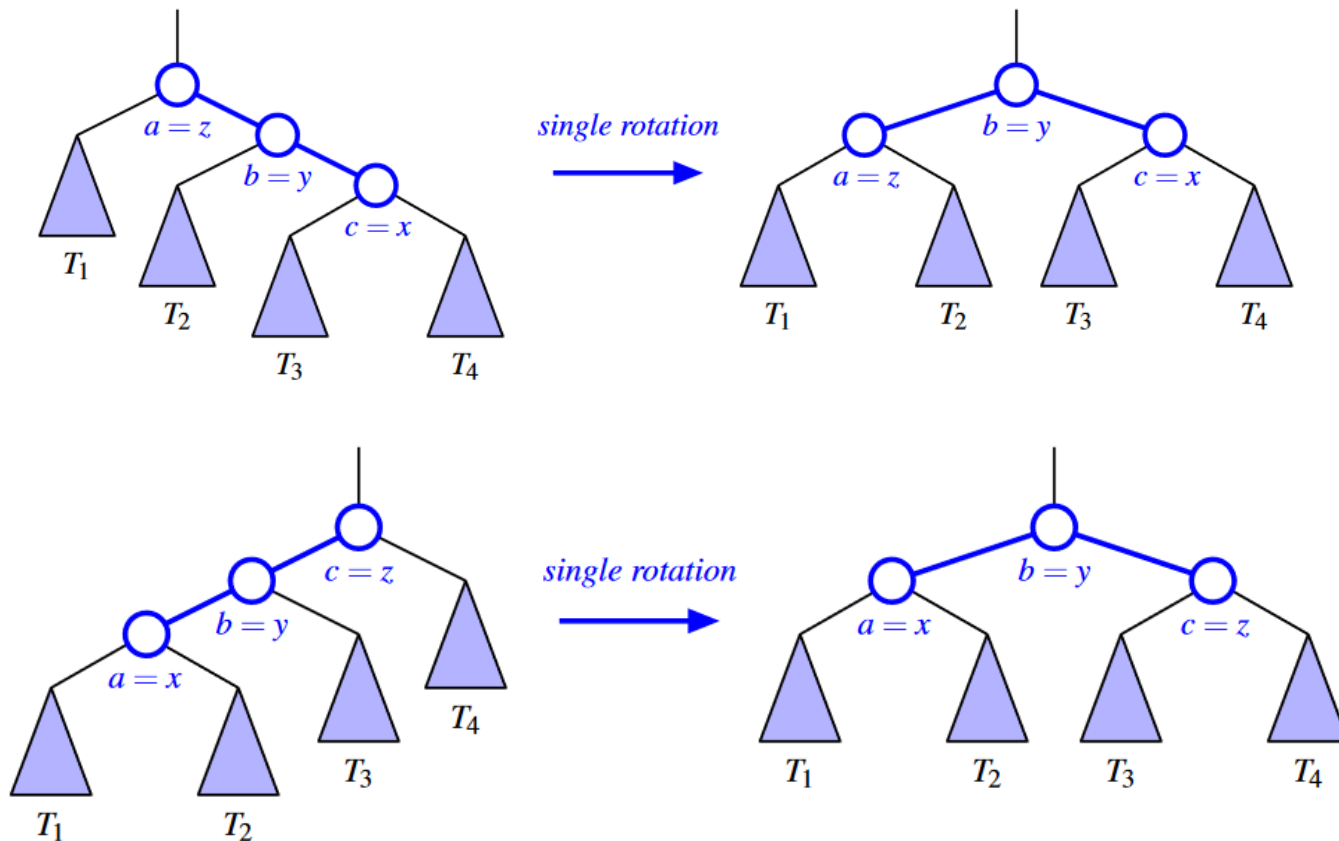
```

y = T.parent(x)
z = T.parent(y)
if (x == T.right(y) and y == T.right(z)) or (x ==
T.left(y) and y == T.left(z)) then
    T.rotate(y)
    return y
else
    T.rotate(x)
    T.rotate(x)
    return x
  
```



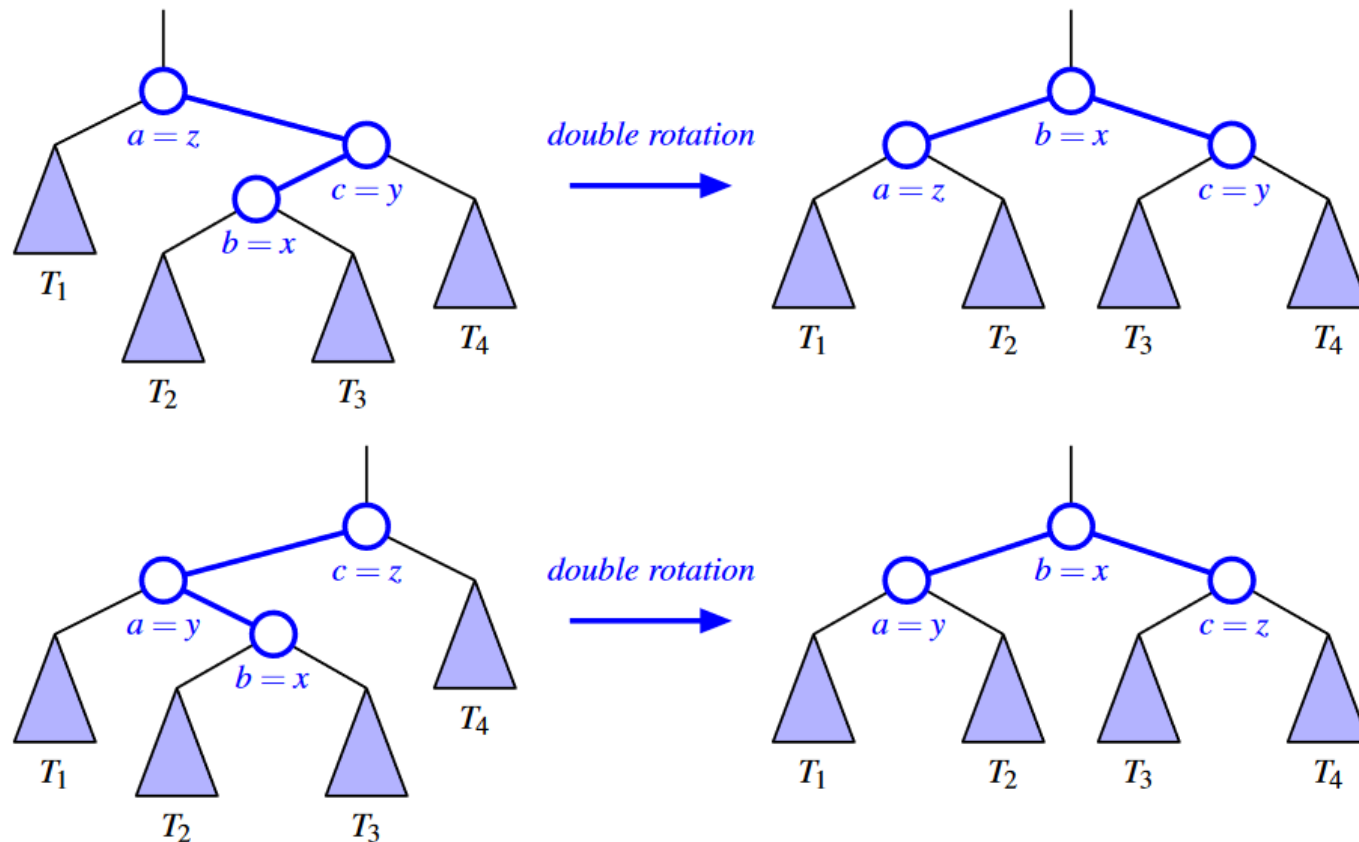
Árbol de búsqueda binaria: árboles balanceados

- Reestructuración trinodo con rotación simple sobre y :
 - Tiempo de ejecución de $O(1)$



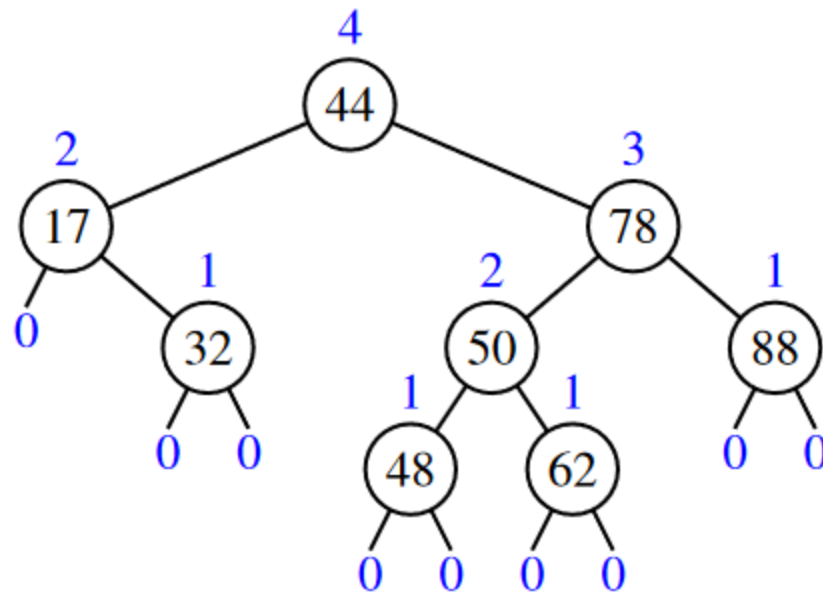
Árbol de búsqueda binaria: árboles balanceados

- Reestructuración trinodo con rotación doble (ocurre cuando x está entre z e y)
 - 1º rota sobre x , y 2º de nuevo sobre x , que ahora ocupa la posición inicial de y
 - Tiempo de ejecución de $O(1)$



Árboles AVL: definición

- Una estrategia para balancear un árbol de búsqueda binaria es añadir una regla que mantenga la altura logarítmica del árbol
- Si consideramos la altura de un nodo como el número de nodos de su camino más largo, la altura de un nodo hoja será 1 y la de sus hijos *None* será 0

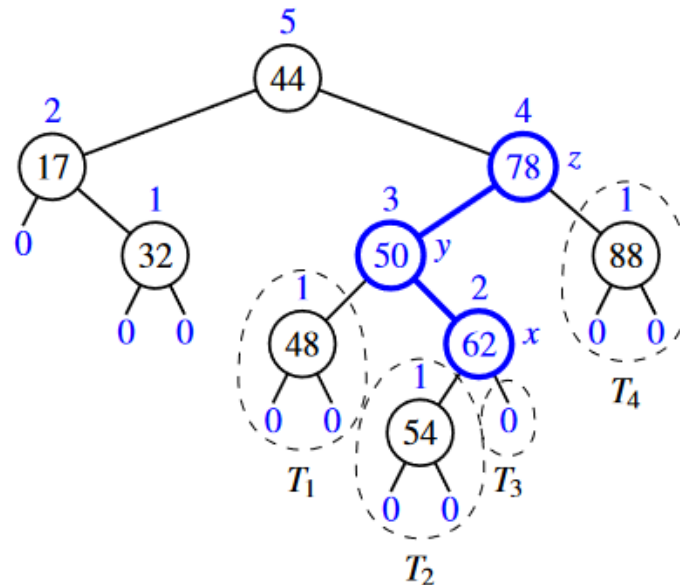


Árboles AVL: definición

- Un árbol de búsqueda binaria T cumple la propiedad de altura balanceada si para cada nodo $node$ de T , las alturas de los hijos difieren como mucho en 1
- Un árbol de búsqueda binaria T que satisface esta propiedad se llama **árbol AVL** (el nombre se debe a las iniciales de sus inventores: Adelson-Velskii y Landis)
- Un árbol AVL mantiene al mínimo la altura del árbol
- Las operaciones que modifican la estructura del árbol (insertar un nodo, borrar un nodo) requieren de operaciones de rotación y reestructuración trinodo para mantener la propiedad anterior
- Un nodo $node$ está balanceado si la altura de sus hijos difiere como mucho en 1. En otro caso está desbalanceado.

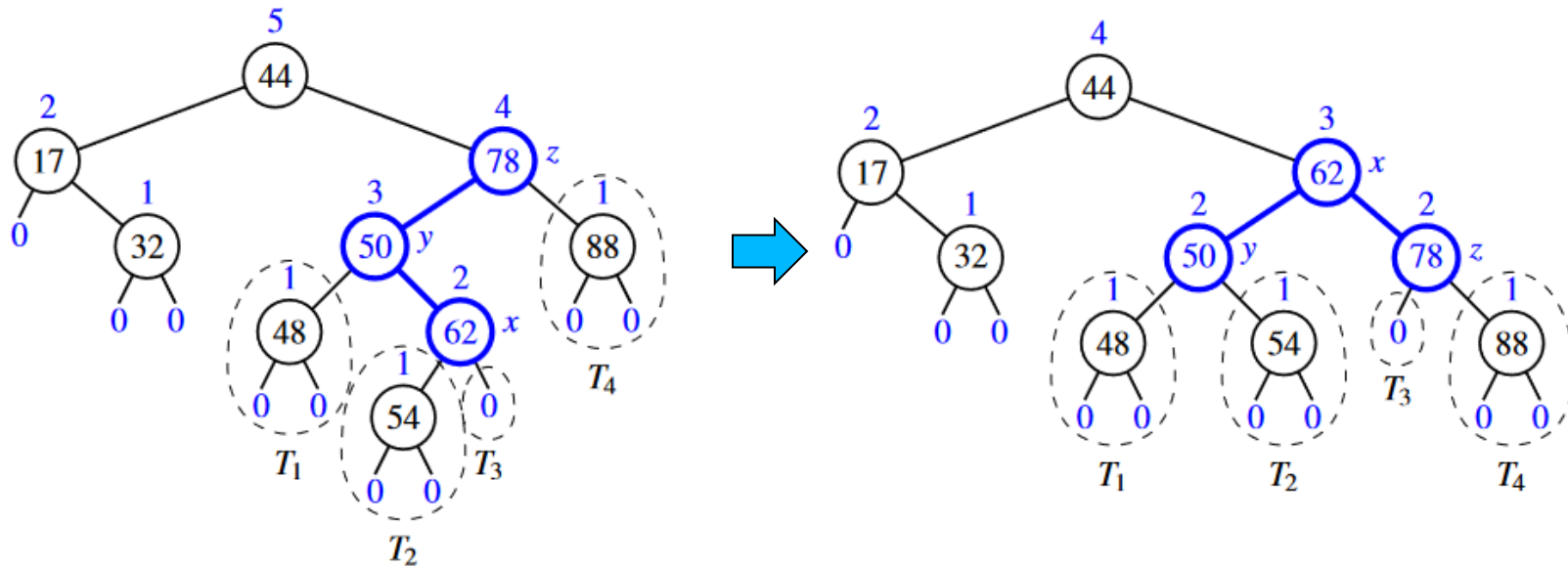
Árboles AVL: operaciones

- La **inserción** de un nodo *node* como hoja en un árbol AVL puede eliminar la propiedad de altura balanceada
- Los nodos desbalanceados solo podrán ser ancestros de *node*
- Ejemplo: Al añadir la clave 54, los nodos 44 y 78 se desbalancean.



Árboles AVL: operaciones

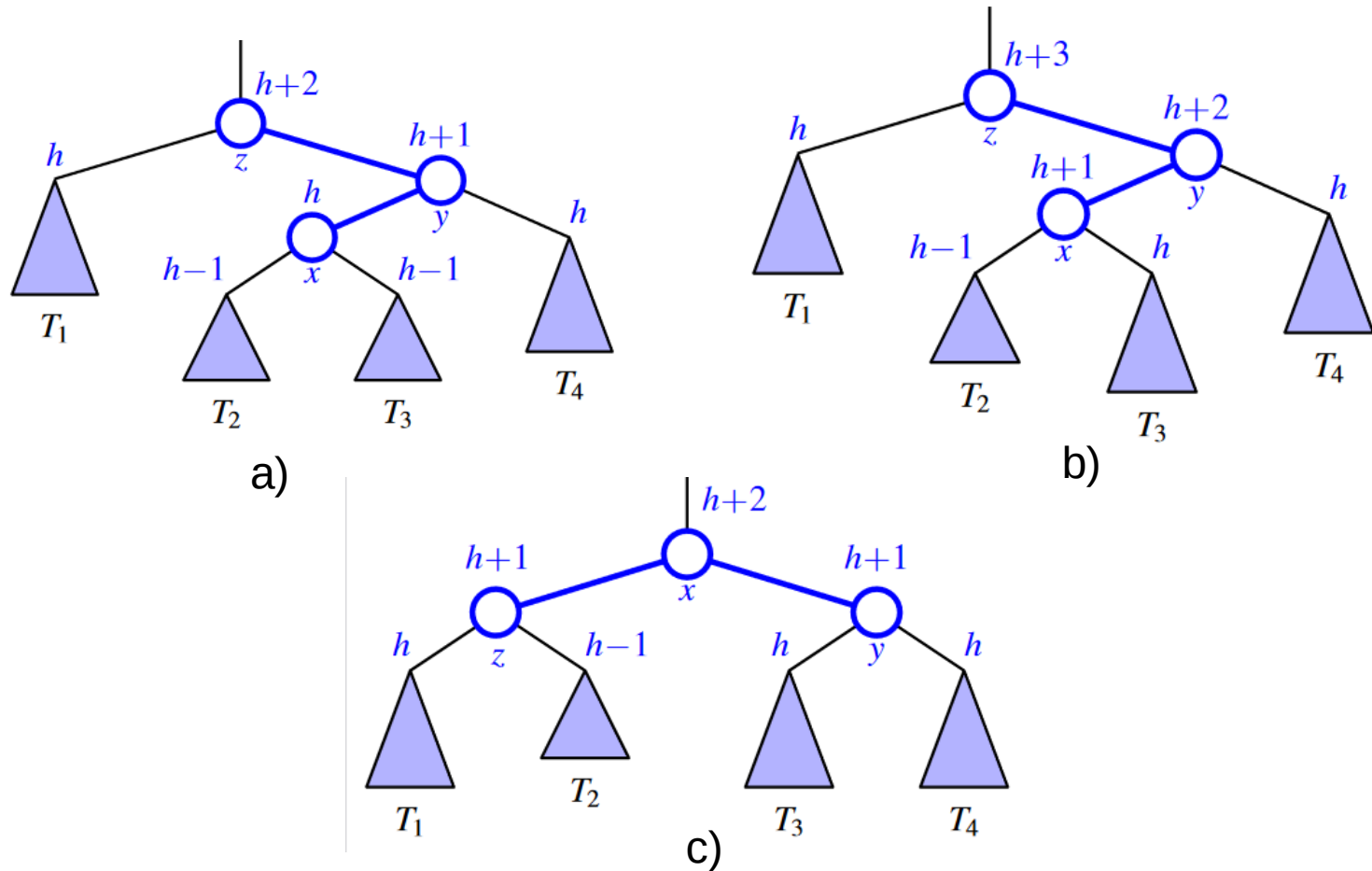
- Una reestructuración trinodo hace que árbol AVL vuelva a estar balanceado



- Identificamos x , y , z de forma que:
 - z es el primer nodo desbalanceado que encontramos subiendo desde *node* (el que acabamos de insertar) a la raíz
 - y es el hijo de z con mayor altura
 - x el hijo de y con mayor altura

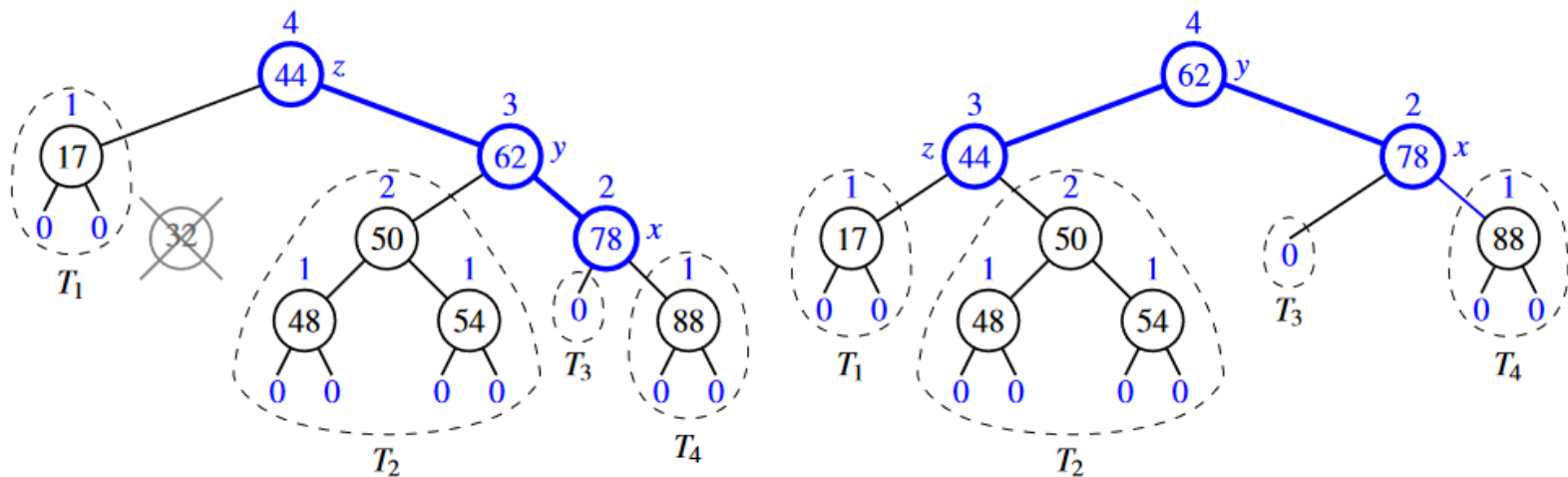
Árboles AVL: operaciones

- Ejemplo: a) antes de insertar; b) tras insertar; c) tras reestructurar



Árboles AVL: operaciones

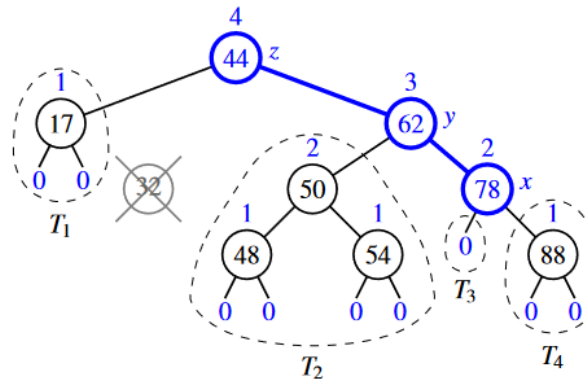
- Al **borrar** un nodo se puede también violar la propiedad de altura balanceada de un árbol AVL
- Si *node* es el padre del nodo borrado, puede haber un nodo (como mucho) desbalanceado en el camino desde *node* a la raíz



- Al borrar el nodo 32, el nodo raíz se desbalancea y con una rotación se vuelve a verificar la propiedad de altura balanceada

Árboles AVL: operaciones

- En la reestructuración trinodo:
 - z será el primer nodo desbalanceado al ascender desde *node* a la raíz
 - y será el hijo de z con mayor altura
 - x será el hijo de y definido como:
 - Si uno de los hijos de y tiene más altura x será ese hijo.
 - Si ambos hijos tienen la misma altura, x será el hijo de y del mismo lado que y (si y es hijo izquierdo, x será el hijo izquierdo de y ; si y es hijo derecho, x será el hijo derecho de y)
- Esta reestructuración puede provocar otros desbalances, por lo que hay que seguir subiendo realizando la operación hasta alcanzar la raíz
- Dado que la altura del árbol es $O(\log n)$, la reestructuración es $O(\log n)$



Árboles AVL: implementación

- Implementación Python en fichero `avl_tree.py`
- Redefine las siguientes operaciones de nuestro árbol binario de búsqueda:
 - `_rebalance_insert(node)`: invocado cuando se añade un nodo *node*
 - `_rebalance_delete(node)`: invocado cuando se elimina un nodo del árbol, siendo *node* el padre del nodo eliminado.
- Se añade un atributo `_height` al nodo definido en la clase `_Node` que contendrá la altura de este, para facilitar las operaciones de comprobación de balanceo

Árboles AVL: eficiencia

- El árbol AVL asegura que todas las operaciones sean de $O(\log n)$, salvo la iteración sobre sus elementos

Operación	Eficiencia
$k \text{ in } T$	$O(\log n)$
$T[k] = v$	$O(\log n)$
$T.\text{delete}(\text{node}), \text{ del } T[k]$	$O(\log n)$
$T.\text{before}(\text{node}), T.\text{after}(\text{node})$	$O(\log n)$
$\text{iter}(T)$	$O(n)$